Chương 5: Quản lý các hoạt động kiểm thử (Test Management) 🎯

5.1. Lập kế hoạch kiểm thử (Test Planning)

FL-5.1.1 (K2) – Mục đích và nội dung của Kế hoạch kiểm thử (Test Plan)

Kế hoạch kiểm thử là tài liệu mô tả **phạm vi, cách tiếp cận, nguồn lực và lịch trình** của các hoạt động kiểm thử.

Nội dung chính thường bao gồm:

- **Phạm vi kiểm thử (Test scope):** Kiểm thử cái gì, không kiểm thử cái gì
- **Cách tiếp cận (Approach):** Kiểm thử thủ công hay tự động, kỹ thuật nào được dùng
- **Nguồn lực (Resources):** Ai tham gia, công cụ gì, môi trường nào
- **Lịch trình (Schedule):** Khi nào bắt đầu, khi nào kết thúc
- **Tiêu chí đầu vào/thoát (Entry/Exit criteria)**
- **Rủi ro và biện pháp giảm thiểu (Risks & mitigations)**

> ***Ví dụ:*** *Bạn chuẩn bị kiểm thử cho ứng dụng đặt vé máy bay. Kế hoạch kiểm thử sẽ ghi rõ: kiểm thử chức năng tìm kiếm chuyến bay, thanh toán, gửi vé qua email; dùng Selenium để kiểm thử tự động; đội gồm 3 tester; kiểm thử từ 1/4 đến 15/4.*

FL-5.1.2 (K1) – Kiểm thử viên đóng góp vào lập kế hoạch Iteration & Release

Kiểm thử viên (tester) không chỉ "ngồi chờ code xong rồi test", mà tham gia **ngay từ giai đoạn lập kế hoạch**:

- **Lập kế hoạch phát hành (Release planning):** Tester giúp xác định phạm vi kiểm thử, ước lượng nỗ lực, đề xuất chiến lược kiểm thử tổng thể cho cả bản phát hành.
- **Lập kế hoạch lần lặp (Iteration planning):** Tester giúp phân tích user story, xác định tiêu chí chấp nhận (acceptance criteria), chia nhỏ user story thành các tác vụ kiểm thử, ước lượng nỗ lực kiểm thử cho từng story.

> ***Ví dụ:*** *Sprint planning cho sprint 5 — tester xem user story "Người dùng có thể đổi mật khẩu" và góp ý: "Story này cần test thêm trường hợp mật khẩu cũ sai, mật khẩu mới quá ngắn, và đổi mật khẩu khi session hết hạn — ước lượng cần 4 giờ kiểm thử."*

FL-5.1.3 (K2) – So sánh Tiêu chí đầu vào (Entry Criteria) và Tiêu chí thoát (Exit Criteria)

Tiêu chí	Entry Criteria (Tiêu chí đầu vào)	Exit Criteria (Tiêu chí thoát)
Ý nghĩa	Điều kiện cần thỏa mãn **trước khi bắt đầu** kiểm thử	Điều kiện cần thỏa mãn **để kết thúc** kiểm thử
Mục đích	Đảm bảo đủ điều kiện để kiểm thử có hiệu quả	Đảm bảo kiểm thử đã đạt mức đủ để chuyển sang giai đoạn tiếp theo
Ví dụ	Môi trường test đã sẵn sàng, test case đã được review, build đã deploy thành công	95% test case đã pass, không còn lỗi nghiêm trọng (critical), báo cáo kiểm thử đã hoàn thành

Trong Agile	Thường gọi là **Definition of Ready**	Thường gọi là **Definition of Done**

> ***Ví dụ thực tế:*** *Trước khi bắt đầu kiểm thử module thanh toán (entry): build 2.3.1 đã deploy lên staging, dữ liệu test đã chuẩn bị. Khi kết thúc (exit): 100% test case critical đã pass, coverage đạt 80%.*

FL-5.1.4 (K3) – Kỹ thuật ước lượng nỗ lực kiểm thử (Test Effort Estimation)

Một số kỹ thuật phổ biến:

Kỹ thuật	Mô tả	Ví dụ
Expert judgment (Đánh giá chuyên gia)	Dựa vào kinh nghiệm của người có chuyên môn để ước lượng	Senior tester nói: "Module này tương tự dự án cũ, cần khoảng 40 giờ"
Historical data (Dữ liệu lịch sử)	Dựa vào dữ liệu từ các dự án tương tự trước đó	Dự án trước có 200 test case mất 5 ngày → dự án này 250 test case ước lượng ~6 ngày
Planning Poker	Cả đội cùng ước lượng bằng lá bài, thảo luận khi có sai lệch lớn	3 tester đưa ra lá 5, 8, 5 → thảo luận người đưa 8, thống nhất là 5 story point
Wideband Delphi	Nhiều chuyên gia ước lượng độc lập, sau đó thảo luận và hội tụ	4 chuyên gia ước lượng: 30h, 40h, 35h, 50h → thảo luận → thống nhất 38h

---

FL-5.1.5 (K3) – Ưu tiên kịch bản kiểm thử (Test Case Prioritization)

Không phải lúc nào cũng đủ thời gian chạy hết tất cả test case, nên cần **ưu tiên** dựa trên:

- **Mức độ rủi ro (Risk level):** Chức năng thanh toán rủi ro cao hơn chức năng đổi avatar → test thanh toán trước
- **Mức độ quan trọng (Business priority):** Chức năng core business được ưu tiên
- **Độ phủ yêu cầu (Coverage):** Ưu tiên test case phủ nhiều yêu cầu nhất
- **Phụ thuộc (Dependencies):** Test login trước rồi mới test các chức năng cần đăng nhập

> ***Ví dụ:*** *Còn 2 ngày trước deadline, 50 test case chưa chạy. Team quyết định: chạy 20 test case rủi ro cao và liên quan thanh toán trước, 15 test case chức năng phụ để sau, 15 test case cosmetic (giao diện) nếu còn thời gian.*

FL-5.1.6 (K1) – Kim tự tháp kiểm thử (Test Pyramid) 🔺

Kim tự tháp kiểm thử mô tả **tỷ lệ phân bổ** các loại kiểm thử theo mức độ:

```
      /\
     /  \       ← Ít test, chạy chậm, chi phí cao
    / UI \         (E2E / System tests)
   /------\
  / Tích   \    ← Số lượng vừa phải
 /  hợp    \      (Integration tests)
```

```
  /------------\
 /  Đơn vị     \  ← Nhiều test nhất, chạy nhanh, chi phí thấp
/_______________\    (Unit tests)
```

Tầng	Loại kiểm thử	Đặc điểm	Ví dụ
Đáy	Unit test (Kiểm thử đơn vị)	Nhiều nhất, nhanh nhất, rẻ nhất	Test hàm `tinhTongGioHang()` trả về đúng tổng tiền
Giữa	Integration test (Kiểm thử tích hợp)	Số lượng vừa phải	Test API thanh toán gọi đúng sang cổng ngân hàng
Đỉnh	UI/E2E test (Kiểm thử giao diện/đầu cuối)	Ít nhất, chậm nhất, đắt nhất	Test toàn bộ luồng: đăng nhập → chọn hàng → thanh toán → nhận email xác nhận

Nguyên tắc: Càng xuống đáy càng nên có nhiều test hơn.

FL-5.1.7 (K2) – Phân vùng kiểm thử (Testing Quadrants)

Testing Quadrants (mô hình của Brian Marick) chia kiểm thử thành 4 góc phần tư dựa trên 2 trục:

- **Trục ngang:** Hỗ trợ đội phát triển (Technology-facing) ↔ Hỗ trợ nghiệp vụ (Business-facing)
- **Trục dọc:** Hướng dẫn phát triển (Guide development) ↔ Phê bình sản phẩm (Critique product)

Quadrant	Hướng	Loại kiểm thử	Ví dụ
Q1 (Technology-facing, Guide)	Hỗ trợ đội dev	Unit test, Component test	Test hàm tính thuế trả kết quả đúng
Q2 (Business-facing, Guide)	Hỗ trợ nghiệp vụ, hướng dẫn phát triển	Functional test, Story test, Prototype	Test user story "khách hàng có thể đặt hàng"
Q3 (Business-facing, Critique)	Phê bình sản phẩm từ góc nhìn nghiệp vụ	Exploratory test, Usability test	Tester thử dùng app như người dùng thật, tìm chỗ khó dùng
Q4 (Technology-facing, Critique)	Phê bình sản phẩm từ góc nhìn kỹ thuật	Performance test, Security test	Test hệ thống chịu được 10.000 người truy cập cùng lúc

---

5.2. Quản lý rủi ro (Risk Management) ⚠️

FL-5.2.1 (K1) – Xác định mức độ rủi ro

Mức độ rủi ro (Risk level) = Xác suất xảy ra (Likelihood) × Tác động (Impact)

Yếu tố	Mô tả	Ví dụ
Risk Likelihood (Xác suất)	Khả năng rủi ro xảy ra	Module mới do dev junior viết → xác suất lỗi **cao**

Risk Impact (Tác động)	Hậu quả nếu rủi ro xảy ra	Lỗi thanh toán → mất tiền khách hàng → tác động **rất cao**
Risk Level (Mức độ)	Likelihood × Impact	Cao × Rất cao = **Nghiêm trọng** → cần ưu tiên test

> ***Ví dụ:*** *Chức năng hiển thị avatar — likelihood lỗi: trung bình, impact nếu lỗi: thấp (chỉ mất hình) → risk level thấp. Chức năng chuyển tiền — likelihood lỗi: trung bình, impact: cực cao (mất tiền) → risk level cao.*

FL-5.2.2 (K2) – Rủi ro dự án vs Rủi ro sản phẩm

Loại rủi ro	Rủi ro dự án (Project Risk)	Rủi ro sản phẩm (Product Risk)
Ảnh hưởng đến	Tiến độ, ngân sách, phạm vi dự án	Chất lượng sản phẩm, trải nghiệm người dùng
Ai quan tâm	Project manager, team lead	Tester, Product owner, end-user
Ví dụ	Tester nghỉ việc giữa chừng, thiếu môi trường test, yêu cầu thay đổi liên tục	Tính năng tính toán sai, hiệu năng chậm, lỗ hổng bảo mật
Cách xử lý	Quản lý nguồn lực, điều chỉnh kế hoạch	Tăng cường kiểm thử ở vùng rủi ro cao

---

FL-5.2.3 (K2) – Phân tích rủi ro ảnh hưởng đến phạm vi kiểm thử

Phân tích rủi ro sản phẩm giúp quyết định:

- **Kiểm thử kỹ hơn (Thoroughness):** Vùng rủi ro cao → test nhiều kỹ thuật hơn, nhiều test case hơn, test sớm hơn
- **Kiểm thử ít hơn:** Vùng rủi ro thấp → test cơ bản, có thể bỏ qua nếu hết thời gian
- **Phạm vi kiểm thử (Test scope):** Rủi ro cao → nằm trong phạm vi bắt buộc; rủi ro thấp → có thể nằm ngoài phạm vi

> ***Ví dụ:*** *Phân tích rủi ro cho app ngân hàng: module chuyển tiền (rủi ro cao) → kiểm thử với 50 test case, bao gồm cả stress test và security test. Module "về chúng tôi" (rủi ro thấp) → chỉ 2 test case kiểm tra hiển thị đúng nội dung.*

FL-5.2.4 (K2) – Biện pháp ứng phó rủi ro sản phẩm

Sau khi phân tích, có thể thực hiện:

Biện pháp	Mô tả	Ví dụ
Giảm thiểu (Mitigation)	Giảm xác suất hoặc tác động bằng kiểm thử	Viết thêm test case cho chức năng rủi ro cao
Chấp nhận (Acceptance)	Chấp nhận rủi ro nếu chi phí xử lý quá lớn	Bug giao diện nhỏ trên trình duyệt ít người dùng → chấp nhận

Chuyển giao (Transfer)	Chuyển rủi ro cho bên khác	Thuê bên thứ 3 kiểm thử bảo mật (penetration test)
Phòng tránh (Contingency)	Chuẩn bị kế hoạch dự phòng	Nếu module thanh toán fail → có kế hoạch rollback

---

5.3. Giám sát, Kiểm soát và Hoàn tất kiểm thử 📊

FL-5.3.1 (K1) – Các chỉ số kiểm thử (Test Metrics)

Chỉ số	Mô tả	Ví dụ
Test case execution rate	Tỷ lệ test case đã chạy / tổng	180/200 = 90% đã chạy
Test case pass rate	Tỷ lệ pass / đã chạy	170/180 = 94% pass
Defect density	Số lỗi / kích thước module	Module A: 15 lỗi / 1000 dòng code
Defect detection rate	Tỷ lệ lỗi tìm được trước release	Tìm 90/100 lỗi trước khi phát hành = 90%
Test coverage	Độ phủ yêu cầu hoặc code	85% requirements đã có test case
Defect trend	Xu hướng số lỗi theo thời gian	Tuần 1: 20 lỗi, tuần 2: 12 lỗi, tuần 3: 5 lỗi → xu hướng giảm, tốt

---

FL-5.3.2 (K2) – Báo cáo kiểm thử (Test Report)

Báo cáo kiểm thử cần được **tùy chỉnh theo đối tượng nhận**:

Đối tượng	Quan tâm gì	Nội dung phù hợp
Đội phát triển (Dev team)	Chi tiết lỗi, vùng nào nhiều lỗi	Danh sách lỗi, mức độ nghiêm trọng, bước tái hiện
Project Manager	Tiến độ, rủi ro, có đúng deadline không	% test case hoàn thành, lỗi blocking, rủi ro còn lại
Khách hàng / Stakeholder	Sản phẩm có sẵn sàng phát hành không	Tóm tắt chất lượng, khuyến nghị Go/No-Go

Có hai loại chính:

- **Test progress report (Báo cáo tiến độ):** Gửi định kỳ trong quá trình kiểm thử
- **Test completion report (Báo cáo hoàn tất):** Gửi khi kết thúc một giai đoạn hoặc dự án

---

FL-5.3.3 (K2) – Truyền đạt trạng thái kiểm thử

Cách truyền đạt phụ thuộc vào bối cảnh:

- **Dashboard (Bảng điều khiển):** Hiển thị trực quan realtime trên công cụ như Jira, TestRail
- **Standup meeting:** Cập nhật nhanh hàng ngày trong Agile — "Hôm qua chạy 20 test case, 2 fail, đang chờ fix"
- **Báo cáo định kỳ:** Email hoặc tài liệu gửi cuối tuần/cuối sprint
- **Biểu đồ (Charts):** Burndown chart, defect trend chart giúp nhìn xu hướng nhanh

---

5.4. Quản lý cấu hình (Configuration Management) 🔧

FL-5.4.1 (K2) – Quản lý cấu hình hỗ trợ kiểm thử

Quản lý cấu hình (Configuration Management - CM) đảm bảo mọi thứ liên quan đến kiểm thử được **kiểm soát phiên bản và truy xuất được**.

CM hỗ trợ kiểm thử bằng cách:

- **Quản lý phiên bản test artifact:** Test case, test data, test script đều được version control
- **Xác định đúng bản build:** Đảm bảo tester đang test đúng phiên bản phần mềm
- **Truy xuất nguồn gốc (Traceability):** Biết test case nào đã chạy trên build nào
- **Tái tạo kết quả:** Có thể quay lại bất kỳ thời điểm nào và chạy lại test với cùng điều kiện

> ***Ví dụ:*** *Tester báo lỗi trên build 3.2.1. Dev nói "đã fix rồi". Nhờ CM, tester kiểm tra build 3.2.2 đúng là có commit fix lỗi đó, và chạy lại đúng test case đã fail trước đó để xác nhận.*

5.5. Quản lý lỗi (Defect Management) 🐛

FL-5.5.1 (K3) – Chuẩn bị báo cáo lỗi (Defect Report)

Một báo cáo lỗi tốt cần đầy đủ thông tin để dev có thể **hiểu, tái tạo và sửa lỗi**:

Thành phần	Mô tả	Ví dụ
ID	Mã định danh duy nhất	BUG-1234
Title (Tiêu đề)	Mô tả ngắn gọn lỗi	"Lỗi thanh toán: Không trừ tiền khi dùng mã giảm giá 100%"
Steps to reproduce (Bước tái hiện)	Các bước cụ thể để gặp lại lỗi	1) Thêm sản phẩm vào giỏ 2) Nhập mã "FREE100" 3) Bấm thanh toán 4) Kiểm tra số dư
Expected result (Kết quả mong đợi)	Điều đáng ra phải xảy ra	Số dư không thay đổi (đơn hàng = 0đ)
Actual result (Kết quả thực tế)	Điều thực sự xảy ra	Số dư bị trừ đúng giá gốc sản phẩm
Severity (Mức nghiêm trọng)	Mức độ ảnh hưởng kỹ thuật	Critical — mất tiền khách hàng
Priority (Độ ưu tiên)	Mức độ cần sửa gấp	High — cần fix trong sprint hiện tại
Environment (Môi trường)	Nơi phát hiện lỗi	Staging, Chrome 120, Android 14

Attachments (Đính kèm)	Ảnh chụp, video, log	Screenshot hiển thị số dư bị trừ sai

> ***Mẹo viết bug report tốt:*** *Viết đủ chi tiết để một người chưa biết gì về tính năng cũng có thể tái hiện được lỗi. Tránh viết mơ hồ như "thanh toán bị lỗi" — hãy nói rõ lỗi gì, ở đâu, khi nào.*

Tóm tắt nhanh Chương 5:

Mục	Nội dung chính
5.1	Lập kế hoạch, ước lượng, ưu tiên, test pyramid, testing quadrants
5.2	Rủi ro = Likelihood × Impact, rủi ro dự án vs sản phẩm, biện pháp ứng phó
5.3	Metrics, báo cáo kiểm thử, truyền đạt trạng thái
5.4	Quản lý cấu hình — version control cho test artifact
5.5	Viết báo cáo lỗi đầy đủ, rõ ràng, tái hiện được

Một số ghi chú

- Tài liệu dịch từ sách gốc ISTQB Foudation v4.0.1 sang tiếng Việt, nhằm mục đích giảm rào cản tiếp cận các kiến thức về testing tới cộng đồng tester Việt Nam nói chung và các anh chị em muốn tìm hiểu về testing nói riêng
- Tài liệu cố gắng dịch nhiều nhất các từ tiếng Việt để bạn đọc không phải tra từ điển trong quá trình đọc (VD: defects dịch là khuyết tật phần mềm).
- Dự án phi lợi nhuận, bạn có thể thoải mái sử dụng bản dịch, chia sẻ và sửa đổi nếu cần thiết.
- Có góp ý cho dự án, bạn có thể submit góp ý qua link này nha: https://go.betterbytesvn.com/sharing-documentation-feedback.
- Bản dịch được thực hiện bởi tác giả Đỗ Minh Phong. Bạn có thể gửi lời cảm ơn/feedback/ donate tới tác giả thông qua các hình thức:
  - Đăng ký kênh Youtube: https://www.youtube.com/@hoctest_com
  - Follow Fanpage: https://www.facebook.com/hoctest/
  - Tham gia group Playwright Việt Nam trên Facebook: https://www.facebook.com/groups/playwright.automation.test
  - Donate cho tác giả: https://academy.betterbytesvn.com/donate-cho-chung-minh/
  - Facebook cá nhân tác giả: https://www.facebook.com/dominhphong.18/

Xin chân thành cảm ơn bạn, vì đã quan tâm tới tài liệu ^^.